

Date: _____

(start)

~~OFFSET~~ → 4 bytes

? also occupies space
Label does not occupy space.

DB

'Ali'

'Sher', 0

1h, 2h, 3h

A	L	i		
S	h	e	r	0
01	02	03		

DW

'AB'

'AB', 0

13ABh, 2FDh

B	A		
B	A	0	0
AB	13	FD	02

DD

'ABCD'

var 'ABCD', 0

'ABC'

'ABC', 0

13ADCDEFh

0AB12h

D	C	B	A				
D	C	B	A	0	0	0	0
C	B	A	0				
C	B	A	0	0	0	0	0
EF	CD	AD	13				
12	AB	0	0				

MOV eax, var+3

data
picked
according to

must
be
compatible

D	C	B	A	0	0	0	0
0	1	2	3				

in eax

at ah

least

if var+3 enters into a data section whose
size doesn't match with eax, no issue.

Date: _____

\$ is not stored in memory, like 'AB'.
Label does not occupy space as well.
align name Label type

align num

- Align totally depends ^{only} on address, not on the values or the type of address.
- The value after align will be affected.
- The value after align, will be stored in the address (nearest) that is divisible by num.

	address	value
var1 db 0Ah	2001	0A
align 10	⋮	→ all NULS in between.
var2 db 0Bh	2010	0B

align word = align 2 etc.

q, o → octal

b, y → binary

d, t, none → decimal

h → hex.

Constants.

e.g. count = 1 or count EQU 1

can be any number or strings.

- They can have arithmetic operations applied to them.

Date: _____

count = 0A

count = count - 1

var db count

09

(not -= or += etc).

count = multi-charactered string can be used for these calculations. but need that proper size like:

(a) $\left\{ \begin{array}{l} \text{count} = 'a' \\ \text{count} = \text{count} - 1 \\ \text{var db count} \end{array} \right.$ X error

(b) $\left\{ \begin{array}{l} \text{count} = 'a' \\ \text{count} = \text{count} - 1 \\ \text{var dw count} \end{array} \right.$ if count-1 goes -ve., the value of complement as.

1 → 108
ascii

count = count - 108

ff	61
----	----

-1 'a'

- These constants donot occupy memory.
- They are treated as 4 bytes (32 bits).
signed (so half of unsigned 4 bytes.
→ 4 characters
or
→ -2147483648 to 2147483648

- ★ Constants cannot be simply used with decimal points. That is a different procedure.
- ★ EQU does not allow redefinition.
- ★ EQU can be used to make aliases for keywords

Like `mv EQU MOV` , `mv ax, 3` ✓ (cl)
 or `mv EQU <MOV ax, v1>`.

- \$ is used to show the current address (4B)
- My address dd \$, This stores the current address in the first four bytes.
- DD is important as address is stored in 4 bytes.

address	value
1234ABCD	CD
← CE	AB
← CF	34
← D0	12
← D1	01
= D2	02

```

myadd dd $
var1 db 01h
var2 db 02h      → address of var 1
∴ so    $ - var1 gives (2)
         current address
         (1234AB D3)
    
```


Date: _____

<>
textequ

Important Point

-ve no.s are stored as 2's comp.
regardless of BYTE or SBYTE.

EQU (general)

name EQU 'ali' Cannot be changed
name EQU 1 //
.....

name EQU <'ali'> Can be changed
name EQU <1> can be changed
allows redefinition. ←

textequ (string literals and numbers as well)
same thing but <> are necessary.

name textequ %(20) 20 is assigned
to name

name textequ <20> ✓
name textequ <'ali'>

EQU w/ <> : Any size of str and number

EQU only : Any size str and 32 bit no.
restrict only in b

textequ : Any size str and number.

Can also be used as in (d)

mv textequ <MOV>

mv textequ <MOV ax, v1>

MOV
-1

Date: _____

Floating Numbers

We use Real4, Real8 and Real10 instead of db, dw, dd etc

val Real4 2.345 can be moved in EAX

val Real8 2.9

Encoding

• It is +ve, so sign bit is 0.

- Done -

- After -

- Loops -

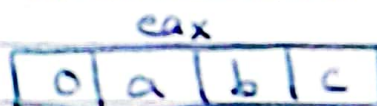
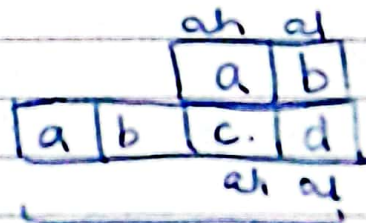


MOV ax, 'ab'

MOV eax, 'abcd'

MOV ax, 'a'

MOV eax, 'abc'



Date: _____

- We can't do $val + val$, like
MOV ax, val + val X
MOV ax, val
ADD ax, val ✓

But

MOV ax, 3 * 3 etc is fine.

MUL

MUL uses ax or eax as the accumulator.

∴ MOV ax, 10

MOV bx, 3

∴ MUL bx ($bx * ax$)

∴ MUL ax ($ax * ax$)

in case of

al → result stored in ax

ax → result stored in DX:AX

eax → result stored in EDX:EAX

MOVSX moves small value in big register.

MOVSX bx, al ✓

val db -3h

MOVSX bx, ax X

⚠ We can't move single constants via movsx

The upper half stores the sign.

like MOVsx ax, val.

ff	fc
ah	al

Date: _____

MOV ax, -4

ff	fc
ah	al

MOV al, -4

0	fc
ah	al

★ ? are stored as 0s.

in MOVZX, higher areas are filled with 0s.

MOVSX ax, val
-16

ff	fo
ah	al

MOVZX ax, val

00	fo
----	----

XCHG : exchanges the values between two registers, or b/w mem and a reg.

- XCHG al, bx X NOT allowed

val1 db 'a'

val2 dd val.

*) val 2 stores the address of val1, so dd is لازم

PTR

PTR can be interpreted as a form of size-casting / typecasting.

var1 dw 1234h

var2 db 0ABh / var3 dw 01ADh

MOV al, BYTE PTR var1

34

MOV ax, WORD PTR var 2

AD AB
ah al

mem. 34 12 AB AD 01

TYPE → tells the bytesize of type
LENGTHOF → total elements.
SIZEOF → total size of var.

In db 'Hello' is 5 bytes (5 elements)
In dd 'Hell' is 4 bytes (1 element)
[0] will be counted in size only.

MOV esi, offset var1
MOV ax, [esi+1]

val LABEL dq X Use QWORD
also.

v1 label dw

v2 dw 01ABh, 02ADh

MOV ax, v1 (v1 is empty, so goes to v2)
... 01 AB
ah al

[] are essential dereferencing.



inc byte ptr [ESI] is incrementing
the value at ESI.

arr1 arr2

Date: _____

same for inc word ptr [ESI]
• Both seem to increment by 1.

```
MOV ESI, 0  
MOV ax, arr[ESI]
```

LOOPS

ECX is the loop counter.

count = 0

loop-start starts loop

LOOP loop-start
check loop

e.g.

```
count db 0  
MOV ECX, 10
```

loop-start:

```
MOV al, count  
INC al
```

```
MOV count, al
```

```
LOOP loop-start
```

→ repeating code.

; count will be 10

★) Doesn't seem to work with constants like count = 0.

JUMPS

We need jumps for inner loops.
Conditional or Unconditional

Like ids and classes
in html!

JMP Jumps to destination Labels immediately.

```
MOV bl, 3
JMP des
MOV al, 2
end:
MOV al, 4
```

4	3
al	bl

Conditional Jumps.

CMP AX, BX

if $(AX == BX)$, $ZF = 0$: JE
ZeroFlag is set

if $(AX \neq BX)$: JNE
ZeroFlag is not set

• **LOOP** → simply decrements register.

if $(AX > BX)$: JG

• **LOOPE / LOOPZ**
also depends on the zero flag.
(terminates at $ECX=0$
 $ZF=0$)

if $(AX \geq BX)$: JGE

if $(AX < BX)$: JL

if $(AX \leq BX)$: JLE

• **LOOPNE / LOOPNZ**
(terminates at $ECX=0$
or $ZF=0$)

Zero Flag : Tells if the result of an operation is 0 or not.

Date: _____

- We use ECX at all cases for loops. In case of inner loops, we just keep saving values.

```
count dw, 0
temp dd, 0
```

```
mov ECX, 3
```

```
outer-loop:
```

```
mov temp, ecx
```

```
mov ecx, 8
```

```
inner-loop
```

} - inner loops code.

```
LOOP inner-loop
```

```
mov ecx, temp
```

```
LOOP outer-loop
```

Inner loops with Jumps.

```
count dw, 0
```

```
mov ECX, 3 → ECX for outer.
```

```
outer-loop:
```

```
mov EDI, 8 → inner loop
```

```
inner-loop:
```

} - inner loop code

DEC EDX $\longrightarrow$ ~~dec~~ dec edx
 JNZ inner-loop $\longrightarrow$ Jump back if edx $\neq$ 0
 LOOP outer-loop.

Encoding.

arr dd 12.5d (if db or dw, it stores 0)

$$\begin{aligned}
 &12d + 0.5d \\
 &1100b + 1000b \\
 &\quad 1100 \cdot 1000b \\
 &1.1001000b \times 2^{\textcircled{3}} \text{ --- index}
 \end{aligned}$$

$$\text{index} + \textcircled{127} = 130$$

$\downarrow$
Fixed value

10000010 (always 8 bit, append 0s to left if not.)

1.1001000b

start till last 1
(second)

8 bits
(first)

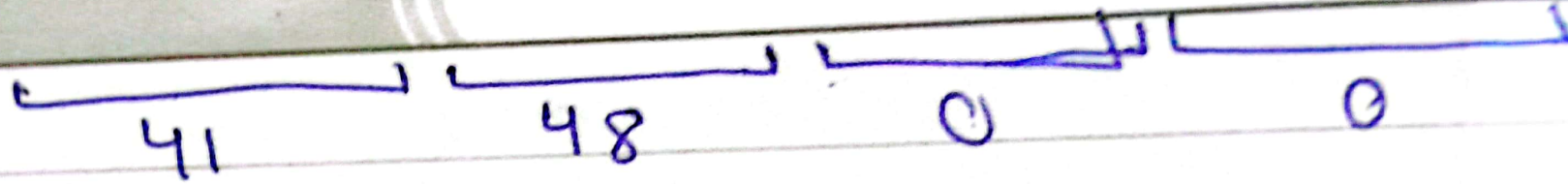
01000000101001b

$\hookrightarrow$ sign bit

- Make it 32 bits by appending 0s to right

01000000101001000000000000000000

Date: _____



by little endian.

